

۱. ماژول Checkpoint Manager

این ماژول که در قالب کلاس `CheckpointManagerPyTorch` پیاده‌سازی شده، وظیفه مدیریت هوشمند ذخیره و بازیابی وضعیت مدل را بر عهده دارد.

- عملکرد `save`:
 - این متد نه تنها وزن‌های مدل (`model.state_dict()`)، بلکه وضعیت بهینه‌ساز (`optimizer.state_dict()`)، تعداد توکن‌های دیده‌شده (`seen_tokens`)، بهترین میزان `loss` اعتبارسنجی (`best_val_loss`) و گام آموزش (`step`) را ذخیره می‌کند.
 - برای هر `Checkpoint` یک نام فایل منحصر به فرد با تاریخ و زمان ایجاد می‌کند تا نسخه‌ها با هم تداخل نداشته باشند.
 - یک قابلیت بسیار مهم به نام `keep_last` دارد که مشخص می‌کند چند `Checkpoint` آخر نگهداری شوند. با ذخیره شدن یک `Checkpoint` جدید، قدیمی‌ترین فایل به صورت خودکار حذف می‌شود تا از پر شدن حافظه جلوگیری شود.
- عملکرد `load_latest`:
 - این متد به صورت خودکار جدیدترین فایل `Checkpoint` را در پوشه مشخص شده پیدا می‌کند.
 - وضعیت مدل و بهینه‌ساز را بارگذاری کرده و آن‌ها را به دستگاه (`GPU/CPU`) صحیح منتقل می‌کند.
 - مقادیر ذخیره شده مانند `seen_tokens` و `step` را برمی‌گرداند تا حلقه آموزش بتواند دقیقاً از همان نقطه‌ای که متوقف شده بود، ادامه یابد.

۲. قابلیت توقف خودکار آموزش

در کد پایه، حلقه آموزش فقط تا زمانی ادامه می‌یابد که به تعداد توکن‌های مشخص شده در `total_tokens` برسد. اما در کد پیشرفته، دو شرط دیگر برای توقف خودکار آموزش در نظر گرفته شده است:

- `max_steps`: حداکثر تعداد گام‌های آموزش.
- `max_time`: حداکثر زمان آموزش به ثانیه.

این تنظیمات در `TrainConfig` قرار دارند و در حلقه اصلی `LLMTrainer` به صورت زیر بررسی می‌شوند:

#بخشی از متد `train` در کلاس `LLMTrainer`

```
if (max_training_steps > 0 and self.step >= max_training_steps) or \
```

```
(max_training_time > 0.0 and (time.time() - start_time) >= max_training_time):
print("Stopping training due to max_steps or max_time reached.")
break
```

این قابلیت به کاربر اجازه می‌دهد تا آموزش را بر اساس محدودیت منابع (زمان یا تعداد تکرار) مدیریت کند.

۳. قابلیت ادامه آموزش از نقطه دلخواه (Resume Training)

این یکی از مهم‌ترین قابلیت‌های اضافه شده است که مستقیماً با Checkpoint Manager کار می‌کند. فرآیند به این صورت است:

۱. پیکربندی: در TrainConfig یک پارامتر resume: bool وجود دارد که به صورت پیش فرض True است.
۲. بارگذاری در LLMTrainer: هنگام ساخت یک نمونه از کلاس LLMTrainer، کدی وجود دارد که این شرط را بررسی می‌کند:

Python

#بخشی از متد __init__ در کلاس LLMTrainer

```
if self.train_config.checkpoint.resume and self.ckpt_manager:
    seen_tokens, best_val_loss, loaded_step = self.ckpt_manager.load_latest(
        self.model, self.optimizer, self.device
    )
    self.seen_tokens = seen_tokens
    self.best_val_loss = best_val_loss if best_val_loss is not None else float('inf')
    self.step = loaded_step
```

۳. نحوه کار:

- اگر resume فعال باشد، LLMTrainer متد load_latest از CheckpointManager را فراخوانی می‌کند.
- این متد، آخرین Checkpoint را پیدا کرده، وزن‌های مدل و وضعیت بهینه‌ساز را بازیابی می‌کند.
- سپس مقادیر seen_tokens و step را از فایل خوانده و برمی‌گرداند.
- LLMTrainer متغیرهای داخلی خود را با این مقادیر به‌روز می‌کند.

نتیجه نهایی این است که وقتی شما اسکریپت را مجدداً اجرا می‌کنید، نوار پیشرفت tqdm و حلقه آموزش از صفر شروع نمی‌شوند، بلکه دقیقاً از همان تعداد توکن و گامی که در اجرای قبلی متوقف شده بودند، کار خود را ادامه می‌دهند. این

ویژگی برای آموزش مدل‌های بزرگ که ممکن است ساعت‌ها یا روزها طول بکشد و احتمال قطعی وجود دارد، ضروری است.